27. Python: Drawing with Turtle Graphics

"Programming that leaves a visible trail behind."

Previous Section:

</> 26. Python: Read and Calculate IP Address

Contents:

Most Python programs work with numbers, text, files, or data. But sometimes, it's much more fun to make the computer draw something.

Python provides the **turtle** library, a simple graphics module that allows us to control a virtual pen on the screen. Instead of plotting pixels manually, we tell the turtle where to move, when to draw, and how to turn.

Think of it as giving instructions to a tiny robot holding a pen:

- Move forward
- Turn left
- Turn right
- Lift the pen
- Put the pen down

By combining these simple actions, we can create shapes, patterns, diagrams, and even small artworks.

Turtle graphics is often one of the first introductions to computer graphics because it teaches an important idea: ***Complex drawings are simply a sequence of small movements.***

Before we start creating shapes, let's look at some of the most common commands you'll encounter.

1. Some Keywords in Turtle

Below are some of the most commonly used methods in the Turtle library.

Method	Description
`forward(distance)`	Move forward while drawing
`backward(distance)`	Move backward while drawing
`left(angle)`	Turn left by an angle
`right(angle)`	Turn right by an angle
`penup()`	Lift the pen (move without drawing)
`pendown()`	Put the pen down (resume drawing)
`goto(x, y)`	Move directly to a coordinate

Method	Description
`circle(radius)`	Draw a circle
`color(color_name)`	Change pen color
`fillcolor(color_name)`	Change fill color
`begin_fill()`	Start filling a shape
`end_fill()`	Finish filling a shape
`speed(value)`	Set drawing speed
`hideturtle()`	Hide the turtle cursor
`clear()`	Clear everything drawn
`done()`	Keep the window open

Most Turtle programs are built using only a handful of these commands. Once you understand how movement and rotation work, you can draw almost anything.

2. Drawing Basic Shapes with Turtle

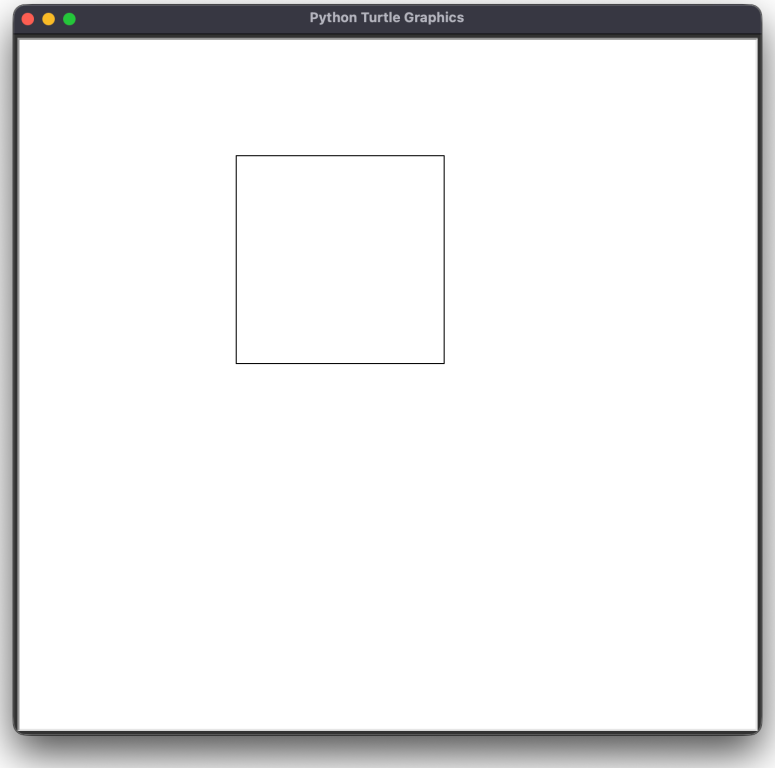
Now let's put those commands into action. The easiest way to learn Turtle is to draw simple geometric shapes and observe how movement and rotation combine to create larger structures.

2.1. Drawing a Square

A square has four equal sides and four 90° corners.

```
import turtle

# Move the pen
turtle.penup()
turtle.goto(-150, 25)
turtle.pendown()

# Drawing
turtle.left(90)
turtle.forward(200)
```

```
turtle.right(90)
turtle.forward(200)

turtle.right(90)
turtle.forward(200)

turtle.right(90)
turtle.forward(200)

turtle.hideturtle()
turtle.done()
```



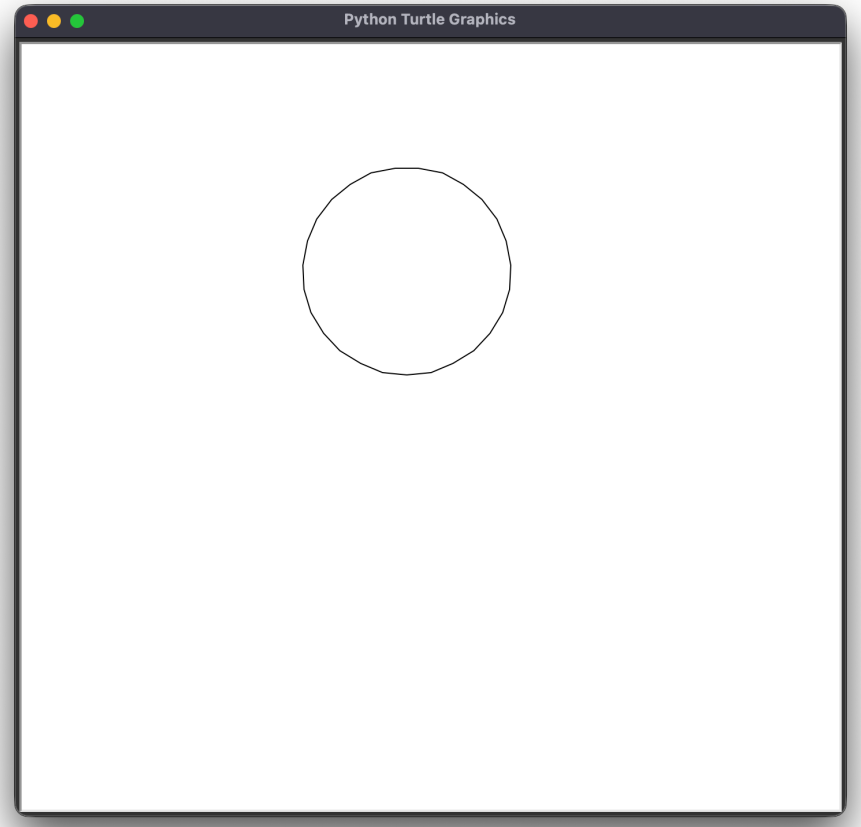
The turtle simply moves forward and turns 90° after each side until the shape is complete.

2.2. Drawing a Circle

Unlike polygons, a circle can be drawn directly using the built-in `circle()` method.

```
import turtle

# Move the pen
turtle.penup()
turtle.goto(-25, 50)
turtle.pendown()

# Drawing
turtle.circle(90)

turtle.hideturtle()
turtle.done()
```

The number inside `circle()` determines the radius of the circle.

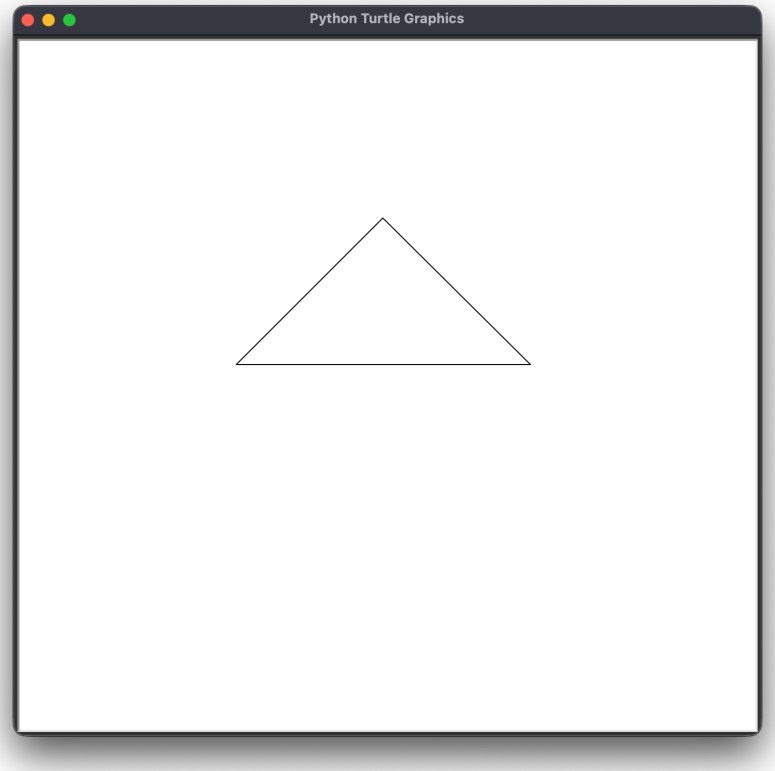


2.3. Drawing a Triangle

Triangles can be drawn by combining forward movement with carefully chosen turning angles.

```
import turtle

# Move the pen
turtle.penup()
turtle.goto(-150, 25)
turtle.pendown()

# Drawing
turtle.left(45)
turtle.forward(200)
```

```
turtle.right(90)
turtle.forward(200)

turtle.right(135)
turtle.forward(282)

turtle.hideturtle()
turtle.done()
```

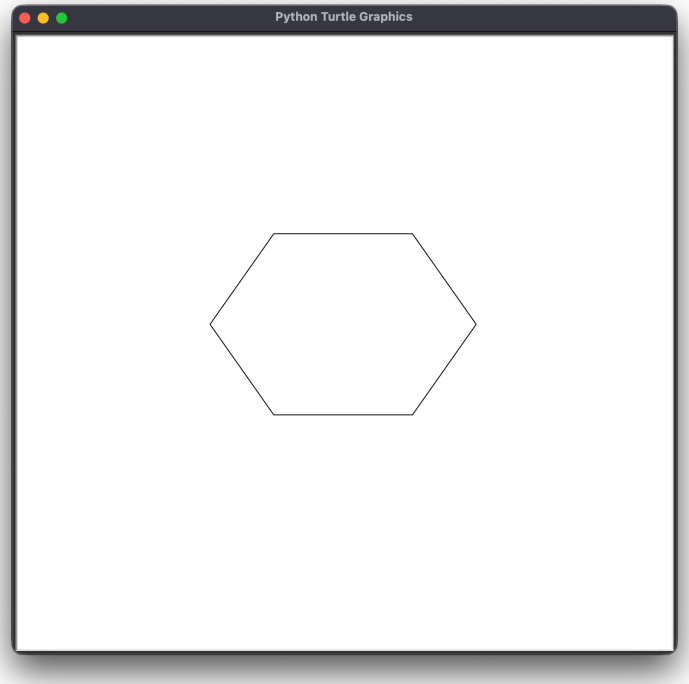


Notice how the final side closes the shape by returning to the starting point.

2.4. Drawing a Hexagon

A hexagon is a six-sided polygon. While there are more efficient ways to draw one using loops, drawing it manually helps us understand how each line and angle contributes to the final shape.

```
import turtle

# Move the pen
turtle.penup()
turtle.goto(-150, 25)
turtle.pendown()

# Drawing
turtle.left(55)
turtle.forward(120)

turtle.right(55)
turtle.forward(150)

turtle.right(55)
turtle.forward(120)

turtle.right(70)
turtle.forward(120)

turtle.right(55)
turtle.forward(150)

turtle.right(55)
turtle.forward(120)

turtle.hideturtle()
turtle.done()
```



Notice something interesting here. A hexagon is really just six connected lines. Once you understand how turning angles work, drawing polygons becomes a matter of geometry rather than memorization.

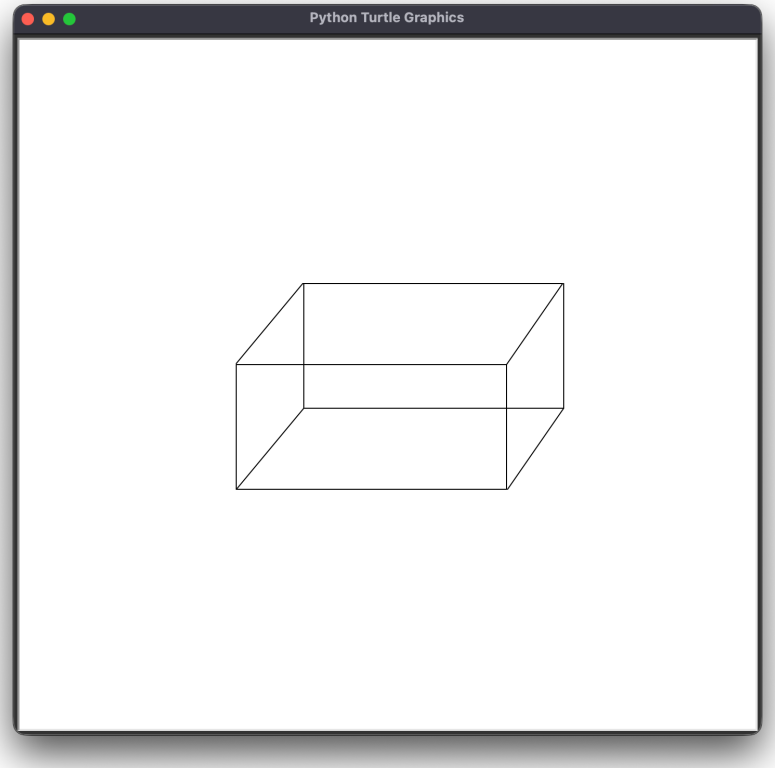
2.5. Drawing a Rectangular Prism (Cuboid)

Turtle is not limited to flat 2D shapes. By carefully combining lines and offsets, we can create the illusion of three-dimensional objects.

```
import turtle

# Move the pen
turtle.penup()
turtle.goto(-150, 25)
turtle.pendown()

# Draw the upper rectangle
turtle.left(50)
turtle.forward(100)

turtle.right(50)
```

```
turtle.forward(250)

turtle.right(125)
turtle.forward(95)

turtle.right(55)
turtle.forward(260)

# First side rectangle
turtle.left(90)
turtle.forward(120)

turtle.left(90)
turtle.forward(260)

turtle.left(90)
turtle.forward(120)

# Second side rectangle
turtle.penup()
turtle.goto(165, 102)
turtle.pendown()

turtle.left(180)
turtle.forward(120)

turtle.right(35)
turtle.forward(95)

# Third side rectangle
turtle.right(145)

turtle.penup()
turtle.goto(-85, 102)
turtle.pendown()

turtle.right(180)
turtle.forward(120)
```

```
turtle.right(40)
turtle.forward(100)

# Final side rectangle
turtle.right(145)

turtle.penup()
turtle.goto(-85, -18)
turtle.pendown()

turtle.right(85)
turtle.forward(250)

turtle.hideturtle()
turtle.done()
```



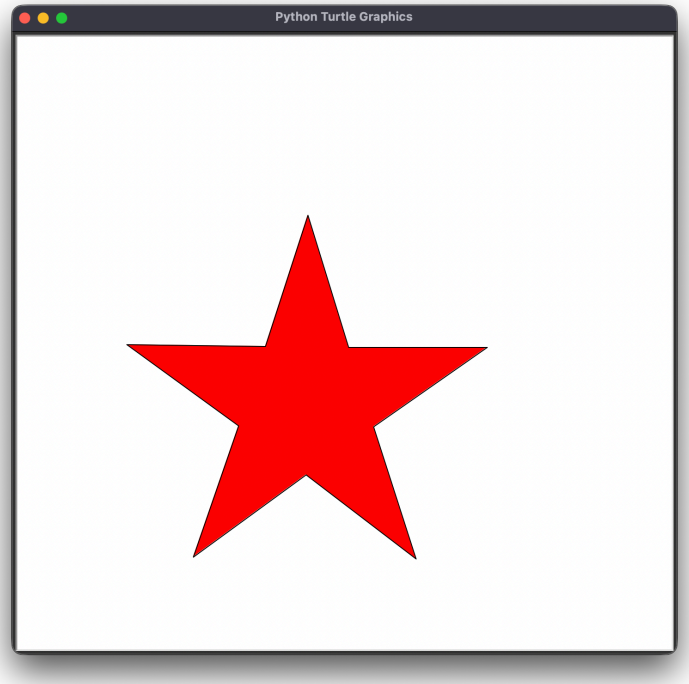
Even though Turtle only draws lines on a flat screen, our brain interprets the connected edges as a three-dimensional cuboid.

2.6. Drawing a Filled Star

Turtle also supports coloring and filling shapes.

```
import turtle

def draw_star(size, color):
    angle = 145

    turtle.fillcolor(color)
    turtle.begin_fill()

    for side in range(5):
        turtle.forward(size)
        turtle.right(angle)

        turtle.forward(size)
        turtle.right(72 - angle)

    turtle.end_fill()

    turtle.hideturtle()
    turtle.done()


size = float(input("Size: "))
color = input("Fill with: ")

draw_star(size, color)
```

Size: 150
Fill with: red



This example introduces two useful methods:

- `begin_fill()`
- `end_fill()`

Everything drawn between these two commands is automatically filled with the chosen color.

By now you've seen the core idea behind Turtle graphics:

> **Every drawing is simply a sequence of movements and rotations.**

Whether you're drawing a square, a star, or a complex geometric pattern, the turtle follows the same principle:

> **Move → Turn → Move → Turn → Repeat**

And surprisingly, that's exactly how many computer graphics systems work underneath as well.

3. Drawing Practical Shapes

So far we've focused on geometric figures: Squares; Triangles; Circles; Hexagons; Stars.

These are useful for learning movement and rotation, but Turtle can do much more. The real fun begins when we combine multiple shapes together to create recognizable objects.

- A clock is just a circle with text and lines.
- A car is simply a collection of circles, rectangles, and angled segments.
- A house combines rectangles and triangles.
- Even decorative artwork such as hearts and fireworks can be built from simple Turtle commands.

In other words: ***Complex drawings are usually many simple drawings working together.***

Let's look at a few examples.

3.1. Drawing a Clock

A clock may seem complicated at first glance. But if we break it down, it consists of only three components:

- A circle for the clock face
- Numbers positioned around the edge
- Two lines representing the hands

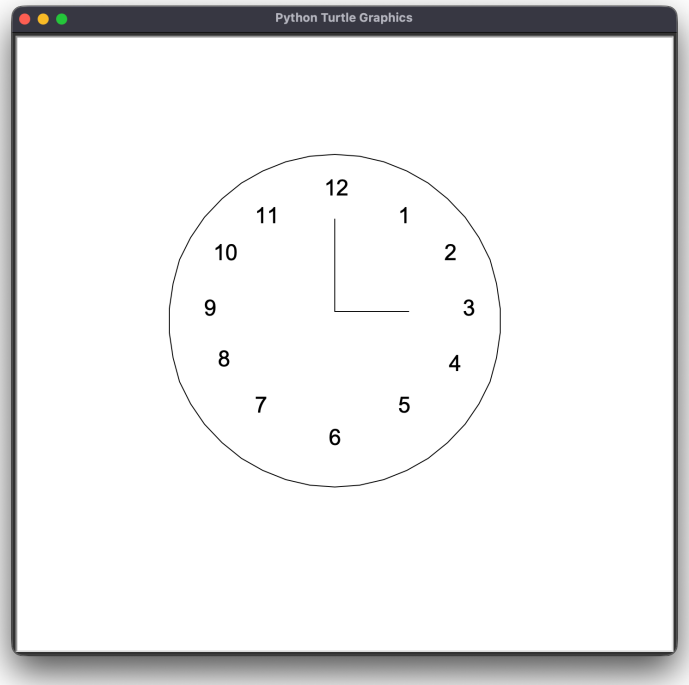
In this example, the clock displays 3:00.

```
import turtle

# Move the pen
turtle.penup()
turtle.goto(-15, -150)
turtle.pendown()

# Drawing a circle
turtle.circle(180)
```

```
# Number in the clock

turtle.penup()
turtle.goto(-25, 160)
turtle.pendown()
turtle.write("12", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(55, 130)
turtle.pendown()
turtle.write("1", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(105, 90)
turtle.pendown()
turtle.write("2", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(125, 30)
turtle.pendown()
turtle.write("3", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(110, -30)
turtle.pendown()
turtle.write("4", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(55, -75)
turtle.pendown()
turtle.write("5", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(-20, -110)
turtle.pendown()
turtle.write("6", font=("Arial", 24, "normal"))

turtle.penup()
```

```
turtle.goto(-100, -75)
turtle.pendown()
turtle.write("7", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(-140, -25)
turtle.pendown()
turtle.write("8", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(-155, 30)
turtle.pendown()
turtle.write("9", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(-145, 90)
turtle.pendown()
turtle.write("10", font=("Arial", 24, "normal"))

turtle.penup()
turtle.goto(-100, 130)
turtle.pendown()
turtle.write("11", font=("Arial", 24, "normal"))

# Show 3:00

turtle.penup()
turtle.goto(-15, 40)
turtle.pendown()
turtle.forward(80)

turtle.penup()
turtle.goto(-15, 40)
turtle.pendown()
turtle.left(90)
turtle.forward(100)
```

```
turtle.hideturtle()
turtle.done()
```



Although the code is long, most of it simply positions numbers around the clock face. This demonstrates an important lesson: ***Graphics often involve placing objects at precise coordinates.***

3.2. Drawing the Olympic Games Logo

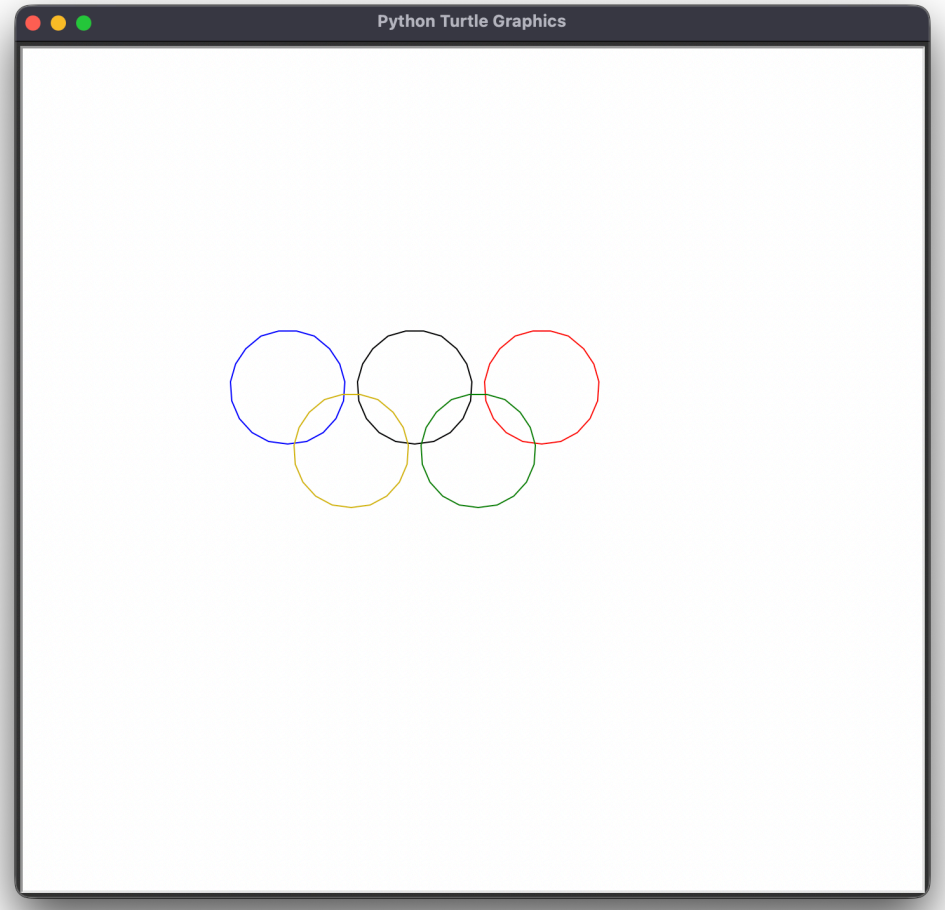
The Olympic logo looks sophisticated, but it is actually made from five overlapping circles.

This example demonstrates:

- Multiple colors
- Positioning shapes
- Reusing the same drawing operation

```
# Draw the Olympic Games logo
import turtle

turtle.color("blue")
turtle.penup()  # penup(): Pick the pen up
turtle.goto(-150, 25)  # goto(): Move the pen to a differen
t position
turtle.pendown()   # pendown(): Mark the pen into the targe
t for drawing
turtle.circle(45)

turtle.color("black")
turtle.penup()
turtle.goto(-50, 25)
turtle.pendown()
turtle.circle(45)

turtle.color("red")
turtle.penup()
turtle.goto(50, 25)
turtle.pendown()
turtle.circle(45)

turtle.color("green")
turtle.penup()
turtle.goto(0, -25)
turtle.pendown()
turtle.circle(45)

turtle.color("#D5B60A")  # Dark Yellow
turtle.penup()
turtle.goto(-100, -25)
turtle.pendown()
turtle.circle(45)

turtle.hideturtle()
```

```
turtle.done()
```



Five circles. Five positions. Five colors.

Sometimes the most recognizable designs come from very simple building blocks.

3.3. Drawing a Simple Car

Now we combine circles and lines to create a complete object.

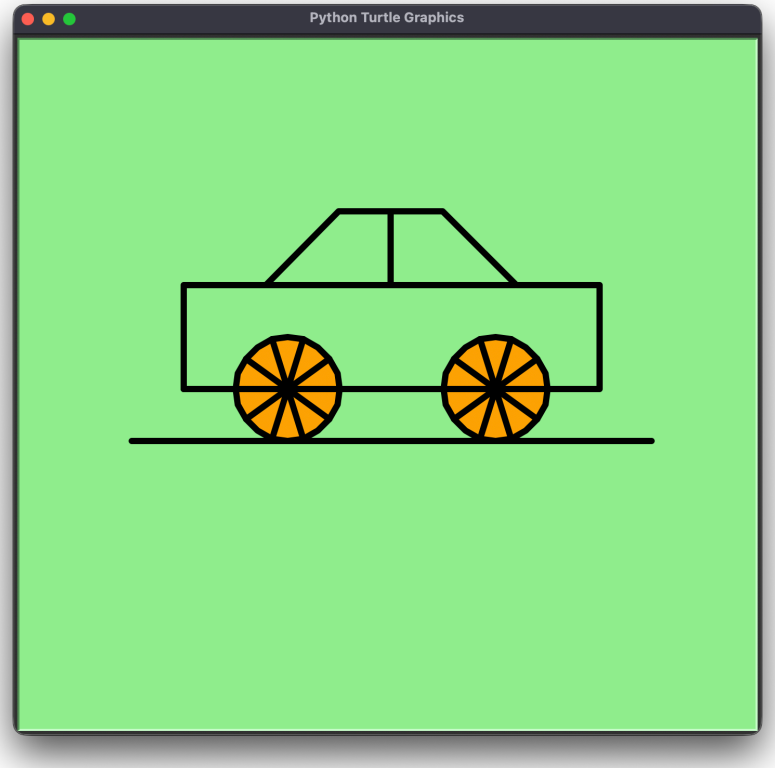
- The wheels are circles.
- The body is a collection of connected line segments.

- The road is simply a straight line.

```
# Draw a simple car
import turtle

screen = turtle.Screen().bgcolor("light green")
turtle.width(6)


def circle():  # Draw a circle
    turtle.pendown()
    turtle.color("black", "orange")
    turtle.begin_fill()
    turtle.circle(50)
    turtle.end_fill()
    turtle.penup()


turtle.penup()
turtle.setx(-200)
turtle.sety(0)
turtle.pendown()
turtle.forward(50)
turtle.penup()
turtle.goto(-100, -50)
circle()
turtle.penup()
turtle.setx(-50)
turtle.sety(0)
turtle.pendown()
turtle.forward(100)
turtle.penup()
turtle.goto(100, -50)
circle()

for i in range(10):  # Draw lines in circle
    turtle.penup()
    turtle.goto(-100, 0)
```

```
    turtle.pendown()
    turtle.left(36)
    turtle.forward(50)

for j in range(10):  # Draw lines in circle
    turtle.penup()
    turtle.goto(100, 0)
    turtle.pendown()
    turtle.left(36)
    turtle.forward(50)

# Draw the body of the car
turtle.forward(50)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
turtle.forward(80)
turtle.right(45)
turtle.forward(100)
turtle.left(45)
turtle.forward(100)
turtle.left(45)
turtle.forward(100)
turtle.left(135)
turtle.forward(120)
turtle.left(90)
turtle.forward(70)
turtle.left(180)
turtle.forward(70)
turtle.left(90)
turtle.forward(120)
turtle.penup()
turtle.goto(-120, 100)
turtle.pendown()
turtle.left(180)
turtle.forward(80)
turtle.left(90)
turtle.forward(100)
```

```
# Draw a road line
turtle.penup()
turtle.goto(-250, -50)
turtle.pendown()
turtle.left(90)
turtle.forward(500)

turtle.hideturtle()
turtle.done()
```



This is one of the most important ideas in computer graphics: **Large objects are often built from smaller primitive shapes.**

Games, animations, and design software all follow this principle.

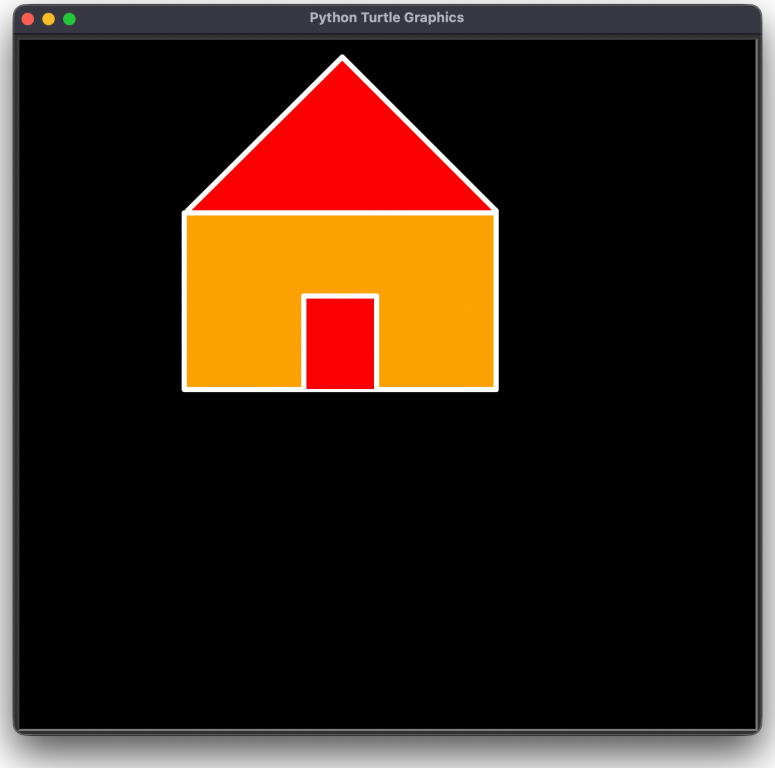
3.4. Drawing a House

A house is another classic example of combining basic geometry. If we look carefully, the entire drawing consists of:

- A rectangle for the walls
- A smaller rectangle for the door
- A triangle for the roof

```
import turtle
import math

screen = turtle.Screen().bgcolor("black")
turtle.speed(18)
turtle.penup()
turtle.goto(-200, 0)
turtle.pendown()
turtle.width(5)
turtle.speed(2)
turtle.color("white", "orange")
turtle.begin_fill()

# Rectangle
for _ in range(2):
    turtle.forward(300)
    turtle.left(90)
    turtle.forward(170)
    turtle.left(90)
turtle.end_fill()

# Door
turtle.forward(115)
turtle.color("white", "red")
turtle.begin_fill()
turtle.left(90)
turtle.forward(90)
turtle.right(90)
turtle.forward(70)
turtle.right(90)
turtle.forward(90)
```

```
turtle.end_fill()
turtle.left(90)
turtle.forward(115)
turtle.left(90)
turtle.forward(172)

# Triangle
turtle.begin_fill()
turtle.left(45)
turtle.forward(210)
turtle.left(90)
turtle.forward(210)
turtle.end_fill()

turtle.hideturtle()

turtle.done()
```



Once again, complex-looking objects become surprisingly simple when broken into smaller pieces.

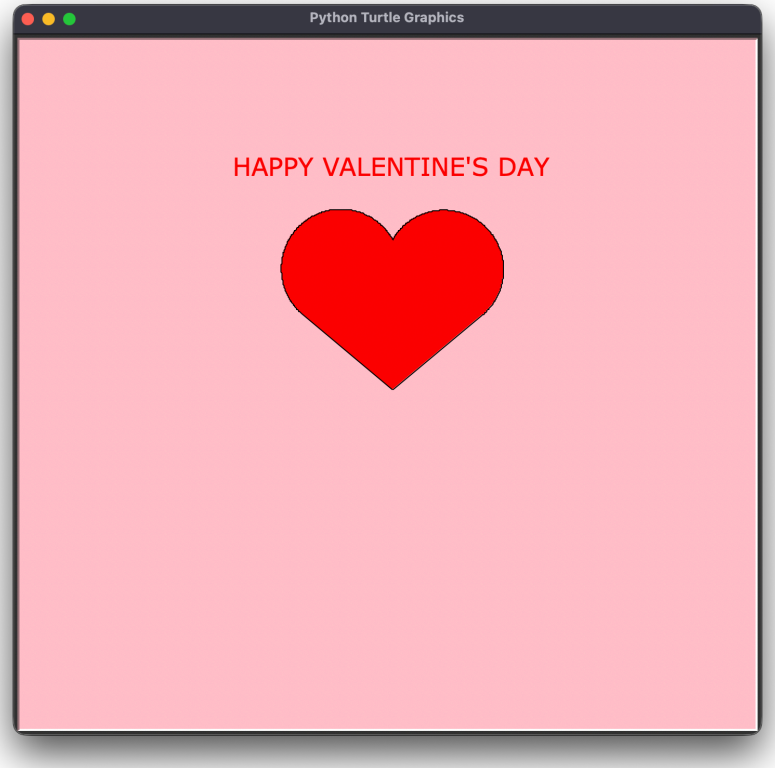
3.5. Drawing a Valentine's Heart

Not every drawing needs to represent a geometric object. Turtle can also create artistic shapes and decorations.

This heart uses a custom curve function to generate smooth rounded edges.

```
import turtle

turtle.bgcolor('pink')


def curve():
    for i in range(200):
        turtle.right(1)
        turtle.forward(1)


def heart():
    turtle.fillcolor('red')
    turtle.begin_fill()
    turtle.left(140)
    turtle.forward(113)
    curve()
    turtle.left(120)
    curve()
    turtle.forward(111)
    turtle.end_fill()


def text():
    turtle.penup()
    turtle.goto(0, 200)
    turtle.color('red')
    turtle.write("HAPPY VALENTINE'S DAY",
```

```
                    align="center", font=("Verdana", 24, "norm
al"))


heart()
text()
turtle.hideturtle()
turtle.done()
```



Here we move beyond geometry and begin creating artwork. The turtle no longer follows straight lines alone—it follows a carefully controlled curve.

3.6. Drawing New Year Fireworks

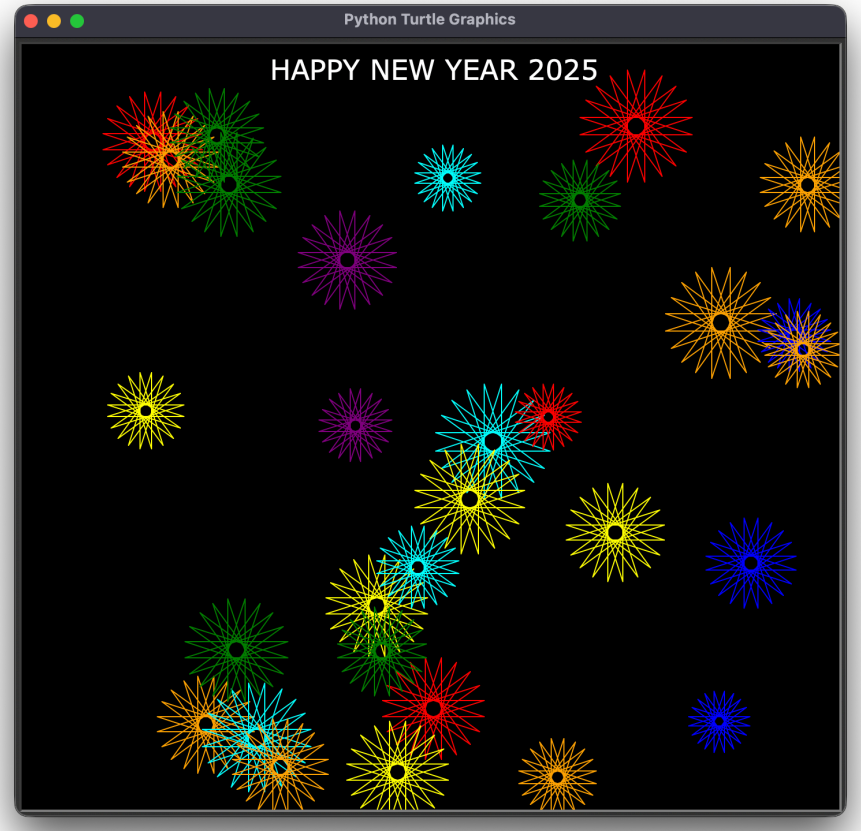
One of the most exciting applications of Turtle is generating random visual effects.

In this example:

- Fireworks appear at random locations
- Random colors are selected
- Multiple explosions are drawn automatically

```
# New Year 2025 Fireworks
import turtle
import random

turtle.Screen().bgcolor("black")


def pen(color):
    turtle.color(color)


def fireworks(distance):
    for _ in range(20):
        turtle.forward(distance)
        turtle.right(180-(360/20))


def move():  # Move the firework in random coordinates
    turtle.penup()
    x = random.randint(-300, 300)
    y = random.randint(-300, 300)
    turtle.goto(x, y)
    turtle.pendown()


def display():
    turtle.penup()
    turtle.goto(0, 300)
    turtle.color('white')
    turtle.write("HAPPY NEW YEAR 2025",
                 align="center", font=("Verdana", 24, "norm
al"))
```

```
turtle.speed(0)
colors = ["red", "purple", "green", "blue", "orange", "yell
ow", "cyan"]
for i in range(30):
    color = random.choice(colors)
    pen(color)
    fireworks(random.randint(50, 100))
    move()

display()
turtle.hideturtle()
turtle.done()
```



Unlike previous examples, the output changes every time the program runs.

This introduces an important programming concept: ***Randomness can make graphics feel dynamic and alive.***

Summary

Turtle is a beginner-friendly graphics library that transforms programming into something visual and interactive.

Throughout this chapter, we learned how to:

- Move a turtle around the screen
- Draw geometric shapes
- Fill objects with color
- Position drawings using coordinates
- Create practical objects such as clocks, cars, and houses
- Generate artistic designs like hearts and fireworks

More importantly, we discovered a fundamental principle of computer graphics: **Complex images are built from simple instructions.**

- A square is just four lines.
- A clock is just a circle and a few numbers.
- A car is simply wheels and connected segments.
- And with enough creativity, those simple movements can become almost anything.

The turtle may be small, but it teaches one of the biggest ideas in programming: **Every masterpiece begins with a single step forward.**

References

https://docs.python.org/3/library/turtle.html

https://www.geeksforgeeks.org/python/turtle-programming-python/

Next Section:

</> 28. Python: Graphical User Interface with Tkinter